

MACHINE LEARNING, TOLD SIMPLY

Book 2 · Contact Session 2

Good Data Before Good Models

The machine learning workflow — understanding data and attributes, fixing quality problems, sampling, scaling, feature engineering, and the vocabulary of learning.

Machine Learning · AIML ZG565 / DSE ZG565 · BITS Pilani WILP

Companion to Contact Session 2 (Module 2: Machine Learning Workflow)

Companion notes prepared by Prof. Saurabh

What is inside

Meera opens the Sahi Deal data and finds it nothing like the tidy tables on the slides. This book is about everything that happens to data before a model ever sees it: knowing what kind of thing each column is, cleaning it, sampling it fairly, putting numbers on a common scale, and building better features. Every formula on the slides is worked with real numbers, and the chapter on hypotheses sets up the decision-tree sessions.

1	What a Dataset Really Is	Objects, attributes, and the three parts every learning algorithm has.
2	Four Kinds of Attribute	Nominal, ordinal, interval, ratio — decided by one question: which arithmetic makes sense?
3	Data Quality: What Can Be Wrong With a Column	Noise, outliers, missing values, duplicates, inconsistencies — how to spot them and what to do.
4	Cleaning by Numbers: Outliers and Imputation	Two rules for deciding "too far": the IQR fence and the 3-sigma fence — worked in full.
5	Aggregation, Sampling and Splitting	Getting less data that still tells the truth — and carving out train, validation and test sets.
6	Feature Scaling	Min-max normalisation, z-score standardisation and decimal scaling — why, when, and the arithmetic.
7	Feature Engineering	Extraction, selection, construction and transformation — turning prepared data into features a model can use.
8	The Vocabulary of Learning	Target function, hypothesis, hypothesis space, version space — and the inductive learning hypothesis that makes all of ML possible.
9	Challenges, Self-Check, and What Comes Next	The three challenges the slides flag for later, a twelve-question self-check, and the bridge to linear regression.

How to read this book

Every chapter follows the same rhythm: a short **story** to set the scene, the **big idea** in one line, an everyday **analogy**, then the **maths step by step** — always the plain version first, and the matrix version only after the plain one is clear. Boxes to watch for:

STORY Meera at Sahi Deal — the running example.	BIG IDEA The one sentence to remember.	THINK OF IT LIKE A real-life picture of the concept.
STOP AND THINK A question to try before reading on.	OUR ANSWER The answer, right after the question.	CAREFUL A common mistake and how to avoid it.
MATHS, STEP BY STEP Every line of algebra, nothing skipped.	WORKED EXAMPLE Real numbers, worked in full.	EXAM TIP How this topic is asked, and how to score.

IN ONE BREATH

The chapter in a few lines.

CHAPTER 1

What a Dataset Really Is

Objects, attributes, and the three parts every learning algorithm has.

STORY

Meera exports the 40,000 listings. Some prices are in rupees and some in lakhs. "Mileage" is empty for a third of the rows. One car has 9,999,999 km on the clock. Two rows are the same Honda City listed twice with different prices. Arjun is not surprised. "Every ML project is 80% this. Let's first agree on what we are even looking at."

ML in a nutshell

The slides open with a sobering fact: there are tens of thousands of ML algorithms, with hundreds more every year. You cannot learn them all — and you do not need to, because **every ML algorithm has the same three components**:

1. Data representation

How do we describe each example as numbers or categories the algorithm can read? (This book.)

2. Parameter optimisation

How does the algorithm set its knobs to fit the data? (Sessions 3–6.)

3. Model evaluation and selection

How do we measure how good it is and choose between candidates? (Sessions 3–6.)

And in practice ML is a **loop**: understand the domain and goals → integrate, select, clean and pre-process data → learn the model's parameters → interpret the results → consolidate and deploy → back to the start with what you learnt.

Objects and attributes

BIG IDEA

A dataset is a **collection of data objects**, and each object is described by a set of **attributes**. An attribute is a property or characteristic of an object — eye colour, temperature, income. Rows are objects; columns are attributes.

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

The tax-cheat table from the slides (it returns in Session 7–8 as the loan-default table). Ten objects, five attributes. "Cheat" is the label.

Because different communities name the same things differently, keep this list handy: an **object** is also called a record, point, case, sample, entity, or instance; an **attribute** is also called a variable, field, characteristic, dimension, or feature.

THINK OF IT LIKE

A school register. Each row is a student (an object). Each column — roll number, name, house, height, marks — is an attribute. A dataset is just a register, and preprocessing is the tidying-up a class teacher does before the data goes to the principal.

Four things that characterise a dataset

Characteristic	Meaning	Why it matters
Dimensionality	The number of attributes	High-dimensional data brings the "curse of dimensionality" (Chapter 8): points become sparse and distances stop meaning much.
Sparsity	Most values are zero or absent; only presence counts	A document as a bag of words: 50,000 possible words, 200 present. Store and compute only the non-zeros.
Resolution	The scale at which data was recorded	Patterns depend on scale: daily sales show weekly cycles, hourly sales show lunch-time spikes, yearly sales show neither.
Size	The number of objects	The type of analysis (and the algorithms that are feasible) depends on how much data there is.

Kinds of data you will meet

The slides list seven types. Most of this course deals with the first; the last two are explicitly out of scope, but you should recognise them.

- **Relational / object (record) data** — a table; one row per object. Our normal case.
- **Transactional data** — each record is a set of items (a shopping basket: {bread, milk, eggs}).
- **Document data** — each document becomes a vector of word counts (a "term vector"); sparse and high-dimensional.
- **Web and social-network data** — a graph: nodes (pages, people) and links.
- **Spatial data** — objects with positions (maps, sensors on a grid).
- **Time series** — one variable measured at successive times (a share price).
- **Sequence data** — an ordered list of symbols (a DNA string, a click-stream).

IN ONE BREATH

Every algorithm = representation + optimisation + evaluation. A dataset = objects (rows) × attributes (columns). Know its dimensionality, sparsity, resolution and size, and which of the seven data types you are holding.

CHAPTER 2

Four Kinds of Attribute

Nominal, ordinal, interval, ratio — decided by one question: which arithmetic makes sense?

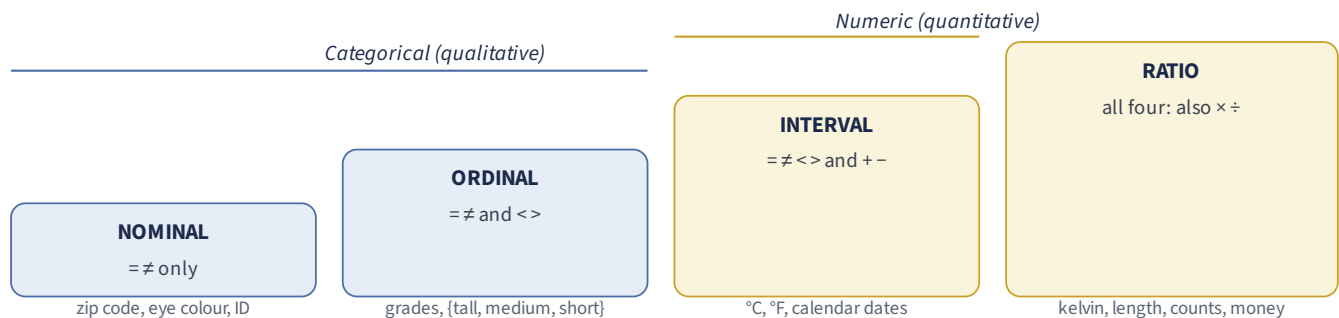
STORY

Meera's spreadsheet has a column "condition" with values Poor, Fair, Good, Excellent. She replaces them with 1, 2, 3, 4 and averages them: "the average condition is 2.7". Arjun raises an eyebrow. "Is Excellent twice as good as Fair? Is the gap between Poor and Fair the same as between Good and Excellent? If not, what does 2.7 mean?"

The test: which operations are meaningful?

Every attribute supports some of these four operations and not others. Which ones it supports decides its type.

Property	Operations	Question it answers
Distinctness	$=, \neq$	Are these two values the same?
Order	$<, >$	Is this value bigger than that one?
Meaningful differences	$+, -$	How much bigger?
Meaningful ratios	$\times, \div$	How many times bigger?



S. S. Stevens's ladder. Each step keeps all the operations of the step below and adds one more.

- Nominal** Values are just **names**. Only "same or different" makes sense. Zip codes, employee IDs, eye colour, sex. Summaries: mode, entropy, contingency tables, χ^2 test.
- Ordinal** Values have an **order** but gaps are not defined. Rankings, grades, hardness of minerals, {good, better, best}, taste on a 1–10 scale. Summaries: median, percentiles, rank correlation, sign tests.
- Interval** Differences are meaningful but there is **no true zero**. Celsius/Fahrenheit temperature, calendar dates. Summaries: mean, standard deviation, Pearson correlation, t and F tests.
- Ratio** A true zero exists, so **ratios** are meaningful. Kelvin temperature, length, mass, counts, elapsed time, money, age. Summaries: geometric and harmonic mean, percent variation.

The temperature puzzle

STOP AND THINK

Is it physically meaningful to say that 10° is *twice* as hot as 5° — on the Celsius scale? Fahrenheit? Kelvin?

OUR ANSWER

Only on the Kelvin scale. Kelvin has a true zero (no molecular motion), so 10 K really is twice 5 K. Celsius zero is just where water freezes — an arbitrary choice — so "twice" is meaningless: 10°C is 283 K and 5°C is 278 K, nowhere near double. Same for Fahrenheit. But *differences* in Celsius are fine: the 5° gap from 5°C to 10°C is exactly as big as from 20°C to 25°C . That is exactly what "interval, not ratio" means.

The height puzzle on the slide is the same trick: "Bill is 3 inches above average, Bob is 6 inches above average" — Bob is *not* twice as tall, because "average" is an arbitrary zero. Height above average is interval; height itself (from the ground, a true zero) is ratio.

Which transformations keep the meaning?

A second way to recognise the type: what can you do to the numbers without changing what they say?

Type	Allowed transformation	Example
Nominal	Any one-to-one relabelling (permutation)	Reassign all employee IDs — nothing changes.
Ordinal	Any order-preserving map $v' = f(v)$, f monotonic	{good, better, best} as {1, 2, 3} or {0.5, 1, 10} — equally valid.
Interval	$v' = av + b$ (scale and shift)	$^\circ\text{F} = 1.8^\circ\text{C} + 32$ — zero moves and the unit changes.
Ratio	$v' = av$ (scale only; zero stays put)	Metres to feet: multiply by 3.28.

Now Meera's 2.7 makes sense — or rather, does not. "Condition" is ordinal. The mapping Poor→1 ... Excellent→4 is only one of infinitely many valid mappings; with Poor→1, Fair→2, Good→3, Excellent→10 the "average" jumps. Report the **median** condition, not the mean.

Discrete versus continuous

This is a different, independent split — about *how many* values, not *what operations*.

Discrete

A finite or countably infinite set of values: zip codes, counts, the words in a document. Usually stored as integers. **Binary** attributes (yes/no) are a special case — and an *asymmetric binary* attribute is one where only the "1" matters (owning an item; the thousands of items you do not own carry no information).

Continuous

Real-number values: temperature, height, weight. Stored as floating-point numbers, which of course means only finitely many digits in practice.

Worked case study: the bank's customer table

The slide asks you to identify the data type and attribute types for a bank that wants to segment customers for targeted marketing.

Name	Gender	Service Rating	Priority?	Card Type	Credit Score	Multiple Accounts?	Income Level	Region
Jack	Male	5	Yes	Platinum	7.5	Yes	Upper	BGLR
Jill	Female	2	Yes	Gold	8.2	No	Middle	DELHI
John	Male	9	No	Gold	7	Yes	Lower	BGLR
Mary	Male	6	No	Gold	6.0	No	Lower	BGLR

WORKED EXAMPLE

Data type: relational / record data — one row per customer.

Attribute	Type	Reason
Name	Nominal (identifier)	A label; only equality makes sense. Useless for prediction — drop it.
Gender	Nominal, binary, symmetric	Two categories, neither is "the presence of something".
Service Rating (1–10)	Ordinal, discrete	9 is better than 5, but 9 is not "1.8 times as satisfied".
Is Priority Customer?	Nominal, binary, asymmetric	"Yes" is the interesting state.
Card Type	Ordinal	Gold < Platinum < Titanium is a real ranking.
Credit Score	Numeric (ratio in form), continuous	Differences meaningful; treat as numeric.
Multiple Account Holder?	Nominal, binary	Yes/No.
Income Level	Ordinal	Lower < Middle < Upper.
Region	Nominal	City codes; no order.

Also notice the data-quality catch: Mary is recorded as Male — probably an error to investigate (Chapter 3).

EXAM TIP

When asked to classify attributes, give the type *and one reason* ("ordinal — ordered but gaps not meaningful"). For borderline cases (a 1–10 rating) state your assumption. Mention whether each is discrete or continuous only if asked — it is a separate axis.

IN ONE BREATH

Nominal: names. Ordinal: names in order. Interval: ordered with meaningful gaps but arbitrary zero. Ratio: a true zero, so "twice as much" makes sense. Ask "which arithmetic is legal?" and the type follows.

CHAPTER 3

Data Quality: What Can Be Wrong With a Column

Noise, outliers, missing values, duplicates, inconsistencies — how to spot them and what to do.

STORY

Meera makes a list of everything odd in the listings: a car with 9,999,999 km; a 2031 model year; mileage blank in 13,000 rows; the same Honda City twice; "Petrol", "petrol" and "PETROL" as three different fuel types; and 2,000 sellers apparently born on 1 January. "Good," says Arjun. "Now you know the data. Most people build the model first and discover this after it fails."

Why quality matters

Poor data quality poisons everything downstream. The slide's example: a loan-risk classifier trained on bad data denies loans to credit-worthy people and approves people who default — a wrong decision in both directions, and the bank loses money on each. Three questions to ask of any dataset: *what kinds of problems are there? how do we detect them? what can we do about them?*

The quality wheel on the slide names six properties of good data: **correct, consistent, interpretable, complete, trustable, usable on demand**. The five typical problems are noise, outliers, wrong or fake data, missing values, and duplicates.

Noise

Noise is the modification of original values by something we do not care about — static on a phone call, "snow" on a TV screen. In a table, it is a temperature recorded as 31.7 when it was really 31.2, or a mileage typed as 18.5 instead of 15.8. For objects (whole rows) noise means an extraneous record that does not belong. The slide's picture of two sine waves plus random jitter says it all: the shape is still there, but every value is slightly off. You rarely remove noise directly; you choose models that are robust to it and average over many examples.

Outliers

Outliers are objects considerably different from most others. There are two very different situations:

Case 1: the outlier is a nuisance

The 9,999,999 km car is a typing error. It will drag a fitted line badly. Detect and remove or correct it (Chapter 4 gives two methods).

Case 2: the outlier is the point

In credit-card fraud or network intrusion, the rare, strange transaction is exactly what we are hunting. Removing outliers would remove the signal.

THINK OF IT LIKE

An unusually tall student in a class photo. If you are measuring the average height of 12-year-olds and she is actually a visiting teacher, she is a nuisance outlier. If you are scouting for the basketball team, she is the one you came for.

Missing values

Values go missing for two reasons: **information was not collected** (people decline to give their age or weight) or **the attribute does not apply** (annual income for a child). Three ways to handle them:

1. **Eliminate** the objects (rows) or attributes (columns) with missing values. Safe if only a few rows are affected; wasteful if a third of the rows are.
2. **Estimate** (impute) the missing value. For a time series of temperatures, interpolate from the neighbouring days. For a census, fill with the mean, median or mode of similar records (same city, same age band).
3. **Ignore** the missing value during analysis — some algorithms (decision trees, for instance) can work around a missing cell.

CAREFUL

Watch for **disguised missing data**: a form that forces a birthday makes everyone born on 1 January; an income field that defaults to 0; a "rating" that defaults to 5. The value is present but meaningless. Detect it by looking for suspicious spikes in a histogram.

Duplicates

When data is merged from different sources, the same object can appear twice, often *almost* identically — one person with two email addresses, one car listed by the dealer and the owner. Duplicates over-weight those objects and, if one copy lands in training and the other in test, they leak the answer. Finding and merging near-duplicates is part of **data cleaning**.

The slide's table, annotated

Tid	Refund	Marital Status	Taxable Income	Cheat	Problem
5	No	Divorced	10000K	Yes	A mistake or a millionaire? Outlier — check the source.
6	No	NULL	60K	No	Missing attribute value.
7	Yes	Divorced	220K	NULL	Missing <i>label</i> — cannot be used for supervised training at all.
9	No	Married	90K	No	Same Tid, different marital status: inconsistent duplicate.
9	No	Single	90K	No	

Inconsistent and intentional errors

The cleansing slide adds two more flavours. **Inconsistent:** Age = 42 but Birthday = 03/07/2010; ratings recorded as 1, 2, 3 in one year and A, B, C in the next; two duplicate records disagreeing. **Intentional:** disguised missing data as above. And the "Petrol/petrol/PETROL" problem is inconsistency of codes — fix by standardising case and spelling before anything else.

MEERA'S CLEANING CHECKLIST

- ✓ Standardise codes and units (one spelling of Petrol; all prices in rupees).
- ✓ Find duplicates and near-duplicates; keep one.
- ✓ Count missing values per column; decide drop / impute / ignore per column.
- ✓ Look for disguised missing values (spikes at 0, at 1 Jan, at "Other").
- ✓ Detect outliers numerically (Chapter 4); decide whether they are nuisance or signal.
- ✓ Check cross-field consistency (age vs birthday; year vs registration).

IN ONE BREATH

Five enemies: noise (values jittered), outliers (values far away — nuisance or signal), missing values (drop, impute or ignore), duplicates (merge), inconsistencies (standardise). Find them *before* modelling.

CHAPTER 4

Cleaning by Numbers: Outliers and Imputation

Two rules for deciding "too far": the IQR fence and the 3-sigma fence — worked in full.

STORY

"How far is too far?" Meera asks. A car with 300,000 km is unusual but real; one with 9,999,999 km is nonsense. She needs a rule, not a feeling. Arjun gives her two — one that uses the middle of the data, one that uses the spread — and a warning that they do not always agree.

Method 1 — the interquartile-range (IQR) fence

BIG IDEA

Sort the data. The middle half of it lies between the first quartile Q_1 (25th percentile) and the third quartile Q_3 (75th percentile). The width of that middle half is the interquartile range $IQR = Q_3 - Q_1$. Anything further than $1.5 \times IQR$ beyond the quartiles is called an outlier:

$$\text{lower fence} = Q_1 - 1.5 IQR, \quad \text{upper fence} = Q_3 + 1.5 IQR.$$

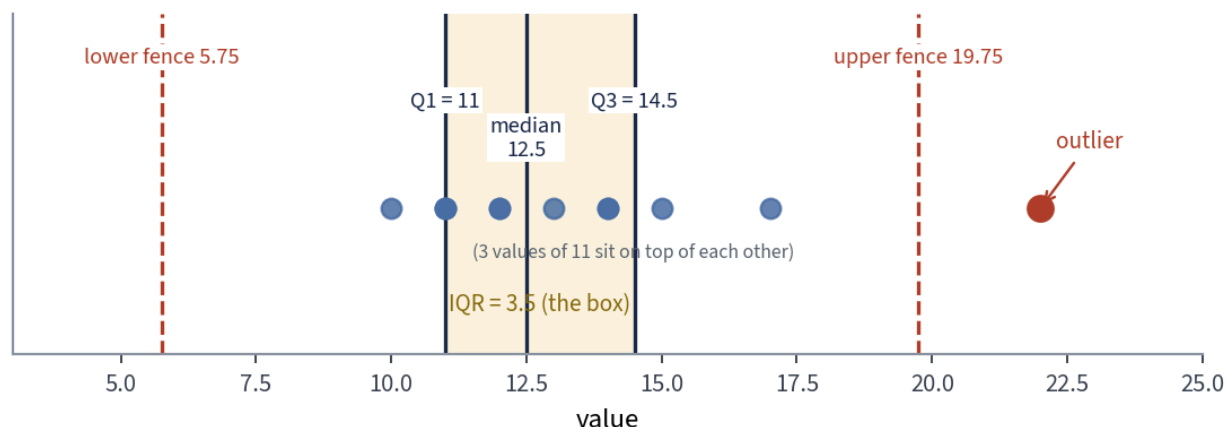
THINK OF IT LIKE

A queue at a ticket counter. The middle half of the queue defines "normal waiting distance". Someone standing one-and-a-half queue-widths beyond the front or the back is clearly not part of the queue. The rule uses the *middle* of the data, so a few wild values cannot move the fence much — that is its strength.

WORKED EXAMPLE (THE SLIDE'S EXERCISE)

Find the outlier in 10, 12, 11, 15, 11, 14, 13, 17, 12, 22, 14, 11 using the IQR.

- 1 **Sort** (12 values): 10, 11, 11, 11, 12, 12, 13, 14, 14, 15, 17, 22.
- 2 **Median** Q_2 : even count, so average the 6th and 7th values: $Q_2 = \frac{12+13}{2} = 12.5$.
- 3 Q_1 = median of the lower half 10, 11, 11, 11, 12, 12: average of the 3rd and 4th: $Q_1 = \frac{11+11}{2} = 11$.
- 4 Q_3 = median of the upper half 13, 14, 14, 15, 17, 22: average of the 3rd and 4th: $Q_3 = \frac{14+15}{2} = 14.5$.
- 5 $IQR = Q_3 - Q_1 = 14.5 - 11 = 3.5$.
- 6 **Fences**: lower = $11 - 1.5(3.5) = 11 - 5.25 = 5.75$; upper = $14.5 + 5.25 = 19.75$.
- 7 **Decide**: only 22 > 19.75. Nothing is below 5.75. **Outlier** = 22.



The twelve values on a line. The box is the middle half; the dashed fences are 1.5 IQR beyond it. Only 22 is outside.

CAREFUL

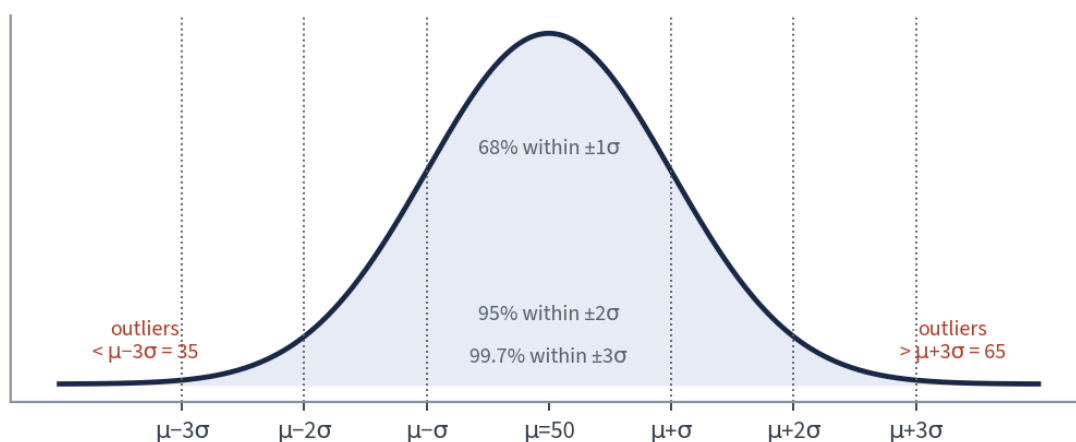
There are several conventions for computing quartiles (software packages differ in how they interpolate). The slides use the *median-of-halves* method shown above — use it in the exam and say so. The slide's own example checks: $Q_1 = 10, Q_3 = 20 \Rightarrow \text{IQR} = 10$, fences -5 and 35 .

Method 2 — the 3-sigma rule

BIG IDEA

If a variable is roughly bell-shaped (normal), then about 68% of values lie within one standard deviation of the mean, 95% within two, and 99.7% within three. So a value beyond $\mu \pm 3\sigma$ is very rare — call it an outlier:

$$\text{lower bound} = \mu - 3\sigma, \quad \text{upper bound} = \mu + 3\sigma.$$



The bell curve with $\mu = 50, \sigma = 5$ (the slide's example). Bounds are 35 and 65; the shaded tails hold only 0.3% of the values.

MATHS, STEP BY STEP — MEAN AND STANDARD DEVIATION

For values $x_1, \dots, x_n$:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}.$$

(The sample version divides by $n - 1$; for outlier screening either is fine — state which you used.) The **z-score** of a value is how many standard deviations it is from the mean: $z = \frac{x - \mu}{\sigma}$. The 3-sigma rule is simply $|z| > 3 \Rightarrow \text{outlier}$.

WORKED EXAMPLE (SLIDE)

$\mu = 50, \sigma = 5$: lower bound $50 - 3 \times 5 = 35$, upper bound $50 + 15 = 65$. Points below 35 or above 65 are outliers. A value of 68 has $z = (68 - 50)/5 = 3.6$ — out.

STOP AND THINK

Apply the 3-sigma rule to the same twelve numbers (10, 11, 11, 11, 12, 12, 13, 14, 14, 15, 17, 22). Is 22 still an outlier?

OUR ANSWER

Sum = 162, so $\mu = 162/12 = 13.5$. Squared deviations:

12.25, 6.25, 6.25, 6.25, 2.25, 2.25, 0.25, 0.25, 0.25, 2.25, 12.25, 72.25; total = 123. Population $\sigma = \sqrt{123/12} = \sqrt{10.25} \approx 3.20$ (sample: $\sqrt{123/11} \approx 3.34$). Upper bound = $13.5 + 3(3.20) \approx 23.1$. So 22 is **not** an outlier by 3-sigma ($z \approx 2.65$). The two rules disagree! The reason: with only 12 points, the value 22 itself inflates σ and pulls the fence outward. The IQR rule, built from the middle of the data, is immune to that. Lesson: the IQR method is more robust for small or skewed samples; 3-sigma assumes a bell shape and plenty of data.

Imputing missing values

When a value is missing and you decide to estimate it, the usual fillers are:

Attribute type	Fill with	Why
Numeric, roughly symmetric	Mean	Keeps the average unchanged.
Numeric, skewed or with outliers	Median	Not pulled by extreme values (income!).
Categorical	Mode (most frequent category)	The only "average" a category has.
Any, with a good grouping variable	Group-wise mean/median/mode	Impute mileage from cars of the <i>same model</i> , not from all cars.
Time series	Interpolation between neighbours	Tuesday's temperature is close to Monday's and Wednesday's.

CAREFUL

Compute the mean/median/mode for imputation **on the training set only** and reuse those numbers on the test set. Otherwise information from the test set leaks into training. The same rule applies to scaling (Chapter 6).

EXAM TIP

IQR problems are pure procedure: sort, median, Q_1 , Q_3 , IQR, two fences, list the outliers. Write every intermediate number — the marks are for the steps. For 3-sigma, state the assumption (approximately normal) and show μ , σ , and both bounds.

IN ONE BREATH

IQR fence: $[Q_1 - 1.5 \text{ IQR}, Q_3 + 1.5 \text{ IQR}]$ — robust, uses the middle. 3-sigma fence: $[\mu - 3\sigma, \mu + 3\sigma]$ — assumes a bell shape. Impute with mean/median/mode, preferably within groups, and fit those numbers on training data only.

CHAPTER 5

Aggregation, Sampling and Splitting

Getting less data that still tells the truth — and carving out train, validation and test sets.

STORY

Meera's laptop chokes on 40,000 rows with 60 columns of derived features. "Use a sample," says Arjun. Meera takes the first 5,000 rows. They are all from January — when only hatchbacks sell. Her sample is smaller, but it is also *wrong*.

Aggregation

Aggregation combines two or more attributes (or objects) into one. Cities aggregated into states; days into months; individual transactions into a monthly total per customer. Three reasons to do it: **data reduction** (fewer rows or columns), **change of scale** (the question is about states, not cities), and **stability** — aggregated data has less variability, because random ups and downs average out. In Python this is a `groupby`: `df.groupby("city")["price"].mean()`.

Sampling

BIG IDEA

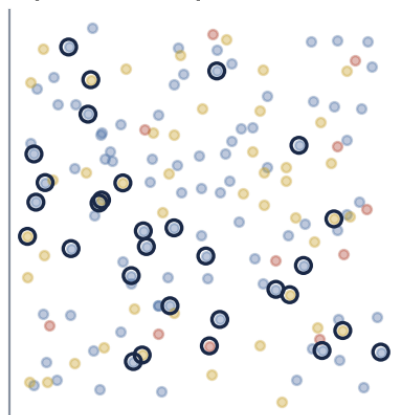
Using a sample will work almost as well as using the entire dataset **if the sample is representative** — that is, if it has approximately the same properties of interest as the whole. Statisticians sample because collecting everything is too expensive; we sample because processing everything is too slow.

Two ways a sample goes wrong, from the slides:

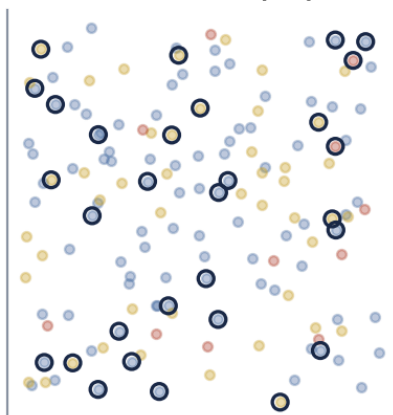
- **Sampling noise** — the sample is too small, so it is randomly unrepresentative. Cure: a bigger sample. The slide's 8000 / 2000 / 500-point pictures show the pattern surviving at 2000 and getting patchy at 500.
- **Sampling bias** — the *process* is flawed, so even a huge sample is unrepresentative (Meera's January rows). Cure: fix the process, not the size.

Three sampling schemes

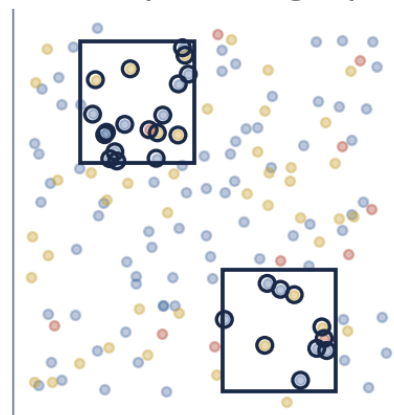
Simple random: pick 30 at random



Stratified: 30 in class proportions



Cluster: pick whole groups



The same population (three classes, 60 / 30 / 10 %) sampled three ways. Circled points are the sample.

Simple random Every object has the same chance. Easy; but a rare class may get few or zero members by chance.

Stratified Split the population into groups (strata — usually the classes), then sample *within each* in proportion. Guarantees every class is represented as in the population.

Cluster Split into natural clusters (villages, branches, days), pick a few clusters at random, and take *everything* in them. Cheap to collect, but the chosen clusters may not represent the rest.

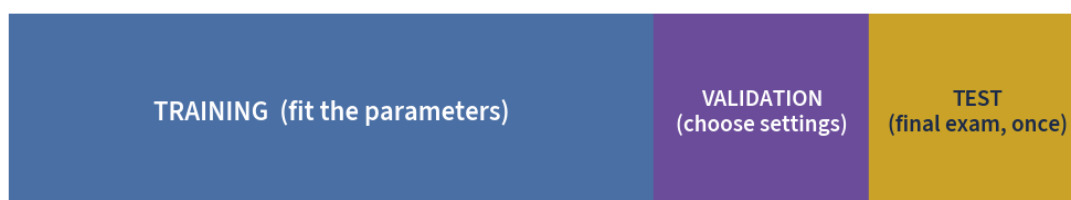
WORKED EXAMPLE — WHY "RANDOM" IS NOT ENOUGH (THE IRIS SLIDE)

The iris data has 150 flowers: 50 setosa, 50 versicolor, 50 virginica. A plain random split into 100 training and 50 test flowers gave: training 38 setosa / 28 versicolor / 34 virginica; test 12 / 22 / 16. Versicolor is under-represented in training (28% instead of 33%) and over-represented in test (44%). A model trained here sees too little versicolor and is then examined mostly on it. **Stratified sampling** would give exactly $\frac{2}{3} \times 50 \approx 33$ of each in training and 17 of each in test.

Imbalanced classes

Sometimes the population itself is lopsided: 1% fraud, 99% honest. A classifier trained on that learns "always say honest". Two standard fixes change the *training* distribution so the rare class is well represented: **under-sample the majority** (throw away many honest rows) or **over-sample the rare class** (repeat, or synthesise, fraud rows — SMOTE, Session 5–6). The test set stays as it is, because it must reflect reality.

Partitioning: train, validation, test



Three slices with three jobs. Typical proportions 60/20/20 or 70/15/15.

Set	Used for	How often it is looked at
Training	Fitting the parameters (the weights of a regression, the splits of a tree)	Every iteration
Validation (evaluation)	Choosing hyper-parameters and comparing candidate models	Many times, while developing
Test	The final, honest estimate of performance on unseen data	Once, at the end

THINK OF IT LIKE

Preparing for a board exam. Textbook exercises = training set. Mock tests = validation set — you use them to decide how to study, which is why doing a mock test a hundred times makes it stop being a test. The board exam = test set. You sit it once, and its score is the one that counts.

CAREFUL

Split **before** any preprocessing that learns from data (imputation values, scaling parameters, feature selection). Fit those on the training set only. And when classes are imbalanced, split with **stratification** so every slice has the same class proportions.

IN ONE BREATH

Aggregate to reduce and stabilise. Sample representatively — stratify when classes matter — and remember that bias is not cured by size. Partition into training (fit), validation (choose), test (final exam, once).

CHAPTER 6

Feature Scaling

Min-max normalisation, z-score standardisation and decimal scaling — why, when, and the arithmetic.

STORY

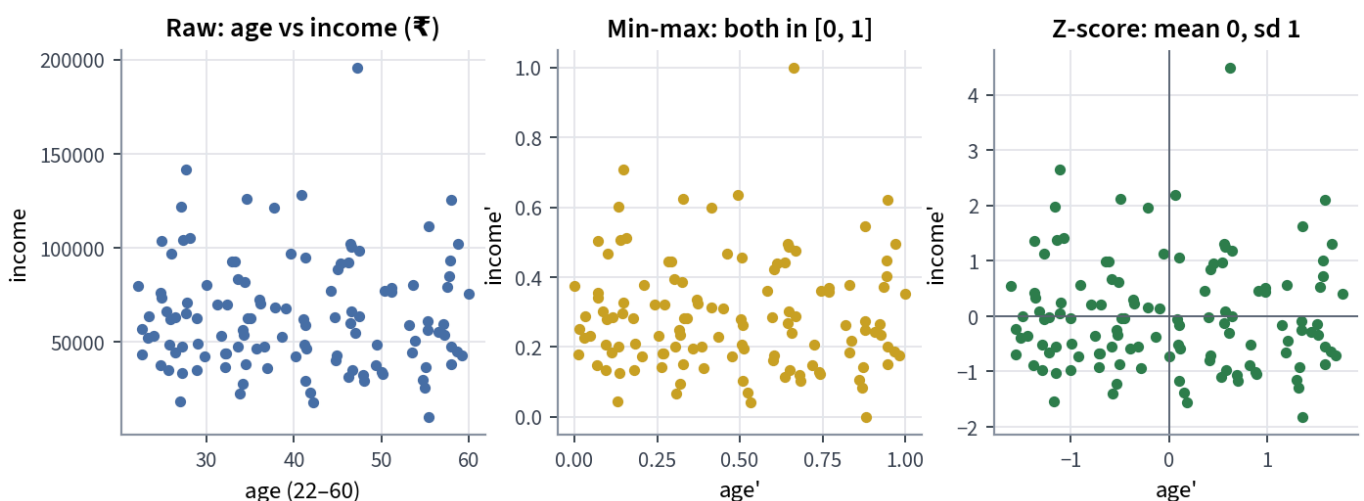
Meera's features: age of car (2–15 years), engine size (800–2500 cc), kilometres driven (5,000–250,000). Her first model behaves as if only kilometres matter. Arjun: "Of course. A step of 1 in kilometres is nothing; a step of 1 in age is a lot. But your algorithm only sees numbers. Put them on the same scale first."

Why scale at all?

Feature scaling maps continuous values from their natural range to a common target range, so that features can be compared and combined sensibly. Two families of algorithms need it badly:

- **Distance-based methods** (k-nearest neighbours, k-means, SVMs): a distance of 1000 in "kilometres" swamps a distance of 5 in "years", so unscaled kilometres decide everything.
- **Gradient-descent methods** (linear and logistic regression, neural networks): with unequal scales the cost surface is a long narrow valley and gradient descent zig-zags slowly (Session 3–4 shows the picture). With scaling the surface becomes round and descent goes straight to the bottom.

Tree-based methods do not care about scale (a split at "km > 80,000" is the same after any monotonic rescaling). And note the slide's remark: **scaling the target y is generally not required.**



The same 120 customers. Raw income dwarfs raw age; after min-max both live in $[0, 1]$; after z-scoring both are centred at 0 with spread 1. The *shape* of the cloud never changes — only the axes.

Min-max normalisation

MATHS, STEP BY STEP

Let the attribute A have minimum $\min_A$ and maximum $\max_A$ in the training data. To map a value v to a new range $[\text{new_min}_A, \text{new_max}_A]$:

$$v' = \frac{v - \min_A}{\max_A - \min_A} (\text{new_max}_A - \text{new_min}_A) + \text{new_min}_A.$$

Read it in two halves. The fraction $\frac{v - \min_A}{\max_A - \min_A}$ says "how far along the original range is v ?" — it is 0 at the minimum and 1 at the maximum. Multiplying by the new width and adding the new minimum places that same fraction inside the new range. For the common target $[0, 1]$ the formula collapses to $v' =$

$$\frac{v - \min_A}{\max_A - \min_A}.$$

WORKED EXAMPLE (SLIDE)

Income ranges from ₹12,000 to ₹98,000; normalise to $[0, 1]$. Where does ₹73,600 go?

$$v' = \frac{73,600 - 12,000}{98,000 - 12,000} (1 - 0) + 0 = \frac{61,600}{86,000} = 0.716.$$

So ₹73,600 is 71.6% of the way from the poorest to the richest. (To $[-1, 1]$ instead: $0.716 \times 2 - 1 = 0.433$.)

Z-score standardisation

MATHS, STEP BY STEP

With the training mean μ_A and standard deviation σ_A :

$$v' = \frac{v - \mu_A}{\sigma_A}.$$

Subtracting the mean centres the attribute at 0; dividing by σ makes its spread 1. The result is "how many standard deviations above the mean" — the z-score from Chapter 4. It is not bounded: a value can be +4 or −7.

WORKED EXAMPLE (SLIDE)

$\mu = 54,000$, $\sigma = 16,000$. Then $v' = \frac{73,600 - 54,000}{16,000} = \frac{19,600}{16,000} = 1.225$. ₹73,600 is 1.225 standard deviations above average income.

Decimal scaling

Move the decimal point: $v' = \frac{v}{10^j}$ where j is the smallest integer such that $\max |v'| < 1$. If the largest absolute value is 98,000, then $j = 5$ and $98,000 \rightarrow 0.98$, $73,600 \rightarrow 0.736$. Crude but quick, and it keeps ratios intact.

Which one, when?

	Normalisation (min-max)	Standardisation (z-score)
Use when	Approximate upper and lower bounds are known and data is roughly uniform across the range (age); the algorithm makes no distributional assumption (kNN, neural nets)	The algorithm assumes roughly Gaussian inputs, or the range is unknown / open-ended
Output range	Bounded: $[0, 1]$ or $[-1, 1]$	Unbounded; mean 0, sd 1
Outliers	One huge value squashes everything else into a tiny interval — avoid on skewed attributes like income	Less affected (though μ and σ do move a little)

CAREFUL — THE GOLDEN RULE OF SCALING

Fit the scaler on the training data only (compute $\min$, $\max$, μ , σ from training rows), then use those same numbers to transform both the training set and the test set. If you recompute on the test set, the test set is no longer "unseen", and a test value outside the training range is exactly the kind of surprise you want to detect, not hide.

STOP AND THINK

A training set has ages 20 to 60. A test customer is 75. What does min-max give? What does that tell you?

OUR ANSWER

$v' = (75 - 20) / (60 - 20) = 1.375$ — outside $[0, 1]$. That is fine and informative: the model is being asked about someone older than anyone it trained on, so its answer deserves less trust. Clipping to 1 would hide that.

IN ONE BREATH

Min-max: $\frac{v - \min}{\max - \min}$, bounded, hates outliers. Z-score: $\frac{v - \mu}{\sigma}$, unbounded, tolerant. Decimal: divide by 10^j . Scale inputs (not the target), fit on training data only, and always for distance- or gradient-based learners.

CHAPTER 7

Feature Engineering

Extraction, selection, construction and transformation — turning prepared data into features a model can use.

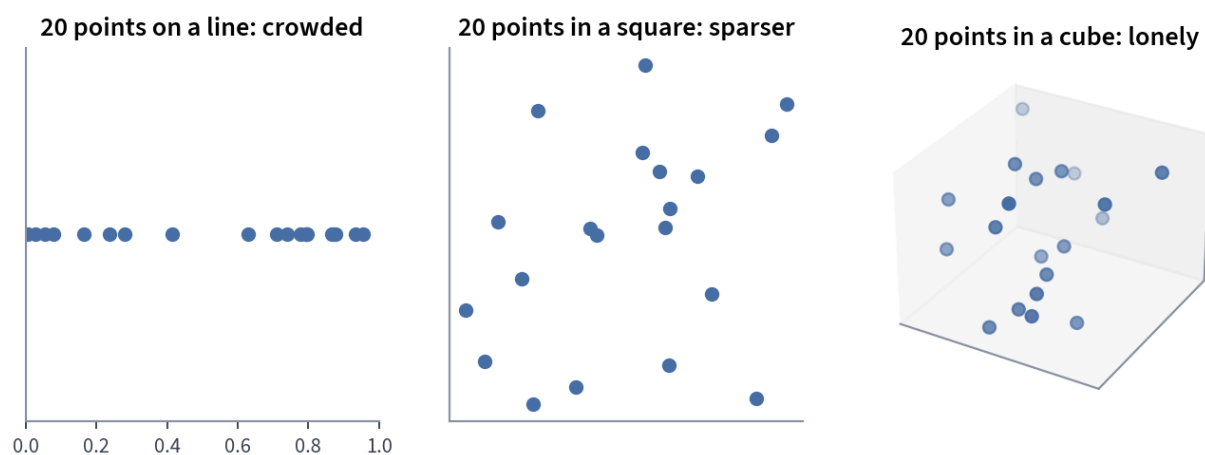
STORY

Meera's table now has 60 columns. Some are useless (the listing ID), some are copies of others (price in rupees and price in lakhs), some are text (fuel type) that a regression cannot read, and the most useful one — *kilometres per year* — does not exist yet. "Now," says Arjun, "you stop cleaning and start engineering."

Data engineering versus feature engineering

The slides draw a line: **data engineering** converts raw data into prepared data (Chapters 3–6: cleaning, sampling, scaling). **Feature engineering** tunes prepared data into the features the model expects. Both are needed because irrelevant, redundant or badly-formatted features make every model worse. There are four kinds of feature engineering.

1. Feature extraction: fewer dimensions



Twenty points on a line, in a square, in a cube. Same number of points, ever emptier space. This is the **curse of dimensionality**.

BIG IDEA

As the number of attributes grows, the data becomes increasingly **sparse** in the space it occupies: every point is far from every other, "nearest neighbour" stops meaning much, and you need exponentially more data to cover the space. Feature extraction fights this by **creating a smaller number of new attributes** that capture most of the information — **Principal Component Analysis (PCA)** is the classic technique.

THINK OF IT LIKE

Describing a person to a sketch artist. You could list 500 measurements. Or you could say "tall, round face, big glasses" — three well-chosen composite features that carry most of what matters. PCA finds those composites automatically as combinations of the original columns.

2. Feature selection: keep the useful columns

Choose a **subset** of the existing features. Drop: **irrelevant** features (Name, DateOfBirth, Section for predicting a GPA), **redundant** features (Age and DateOfBirth say the same thing; price in rupees and in lakhs), and features **missing most of their values**. In pandas: `df.drop(["COLNAME-1", "COLNAME-2"], axis=1)`. Fewer features means faster training, less overfitting, and a model you can explain.

3. Feature construction: build better columns

Create new features from old ones:

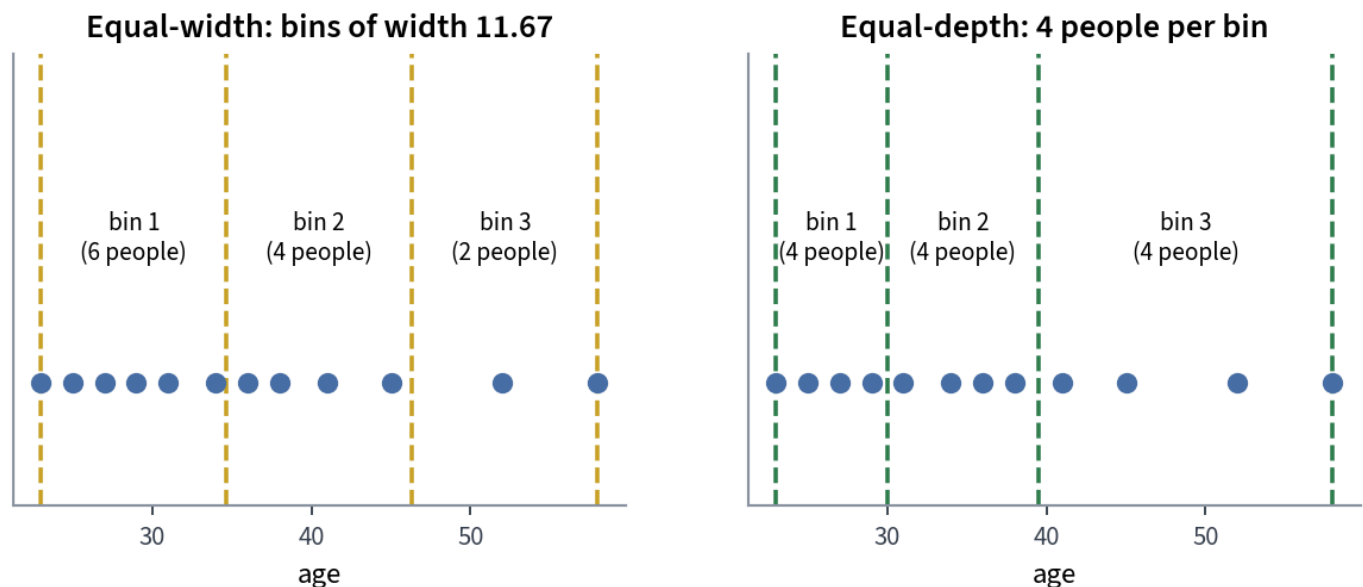
- **Polynomial expansion** — from x make x^2 , x^3 , $\sqrt{x}$, $\log x$ (a univariate function of one column). This is how a *linear* model fits a *curve* (Session 3–4).
- **Feature crossing** — from x_1 and x_2 make $x_1 x_2$ to capture an interaction (engine size $\times$ age).
- **Business logic** — kilometres $\div$ age = kilometres per year; number of semesters completed; days since last service. The best features usually come from domain knowledge, not from formulas.

4. Feature transformation: change the representation

The slide's three-column picture shows one attribute evolving: *SEM-1-Total* (a raw total) $\rightarrow$ *SEM-1-GPA* (rescaled) $\rightarrow$ *S1-isComplete* (a yes/no flag). Same information, three representations, each right for a different question. Two transformations matter most in this course.

Discretisation (binning) of numeric features

Convert a continuous attribute into a discrete one: age → intervals (0–10, 11–20, ...) or concept labels (youth, adult, senior). Why? Naive Bayes, decision trees and their ensembles, and nearest-neighbour methods often prefer discrete inputs; binning also tames outliers (everything above 100 is just "the top bin") and can improve the spread of values.



Twelve ages, three bins, two rules. Equal-width slices the *range* equally; equal-depth slices the *people* equally.

MATHS, STEP BY STEP — THE TWO BINNING RULES

Equal-width (distance) partitioning. If A and B are the lowest and highest values and you want N intervals, each has width $W = \frac{B - A}{N}$. Bin k covers $[A + (k - 1)W, A + kW)$. Simple and uniform, but outliers stretch the range and skewed data piles up in one bin.

Equal-depth (frequency) partitioning. Sort the values and cut so that each of the N bins holds about n/N objects. Handles skew well; bin edges fall wherever the data is dense.

WORKED EXAMPLE

Ages: 23, 25, 27, 29, 31, 34, 36, 38, 41, 45, 52, 58 ($n = 12$), $N = 3$ bins.

Equal-width: $W = (58 - 23)/3 = 11.67$. Edges 23, 34.67, 46.33, 58. Bin 1 [23, 34.67): 23, 25, 27, 29, 31, 34 — six people. Bin 2 [34.67, 46.33): 36, 38, 41, 45 — four. Bin 3 [46.33, 58]: 52, 58 — two. Uneven, because ages are skewed young.

Equal-depth: $12/3 = 4$ per bin. Bin 1: 23, 25, 27, 29; bin 2: 31, 34, 36, 38; bin 3: 41, 45, 52, 58. Cut points can be placed midway between neighbours: 30 and 39.5.

Encoding categorical features

Most algorithms want numbers. Two ways to turn "Fuel = Gas / Diesel" into numbers:

One-hot (dummy) encoding — binarisation

Car	Fuel	Gas	Diesel
A	Gas	1	0
B	Diesel	0	1
C	Gas	1	0
D	gas	1	0

One binary column per category. No false ordering is introduced. Use for **nominal** attributes and for linear / distance-based models.

Label encoding

Fuel	Code
Gas	1
Diesel	2
Gas	1
gas	3

One integer per category. Compact, but it invents an order (Diesel > Gas?). Fine for **ordinal** attributes and for trees; risky for regression. Note the trap in row D: "gas" got a new code because of a spelling inconsistency — clean first!

Case study: the app-feedback table (Problem Type 3)

A marketing team collected feedback on an app from students. The slide asks: *identify the basic preprocessing and data cleaning required.*

Features liked	Features to improve	Similar app?	Pay per month	Rate game (1–10)	Social connect (1–5)	Shopping (1–10)	Recommend?	Education
Graphics	Usability	No	\$35	8	5	4	High	School
Interactive	More features	—	Rs.500	8	3	6	Low	College
Graphics	—	Yes	Rs.250	7	5	6	Medium	School
Cheap	Creativity	Yes	\$20	7	5	8	High	College

WORKED ANSWER

- Missing values:** "Similar app?" is blank in row 2; "Features to improve" blank in row 3. Impute with the mode ("Yes") or add an "unknown" category; or drop if the analysis needs it.
- Inconsistent units:** "Pay per month" mixes dollars and rupees. Convert everything to one currency (say ₹ at a fixed rate) and store as a number, not text.
- Free-text categories:** "Features liked / to improve" are open text — standardise spelling and case, then one-hot encode (Graphics, Interactive, Cheap ...).
- Ordinal encodings:** Recommend (Low < Medium < High) and Education (School < College) are ordinal — label-encode 1, 2, 3 preserving order.
- Binary:** "Similar app?" Yes/No → 1/0.
- Scale mismatch:** ratings are on 1–10 and 1–5 scales; min-max normalise all ratings to [0, 1] before any distance-based analysis.
- Outliers / sanity:** check that ratings lie in their allowed ranges and payments are non-negative.

IN ONE BREATH

Extract (PCA: fewer, denser dimensions), select (drop irrelevant/redundant), construct (powers, crosses, domain formulas), transform (bin numerics — equal-width $W = (B - A)/N$ or equal-depth n/N each; encode categoricals — one-hot for nominal, label for ordinal).

CHAPTER 8

The Vocabulary of Learning

Target function, hypothesis, hypothesis space, version space — and the inductive learning hypothesis that makes all of ML possible.

STORY

Before Meera fits anything, Arjun asks her to read Chapter 2 of Mitchell. "The textbook uses words like *hypothesis* and *version space*, and the decision-tree sessions assume you know them. Learn the words now, on a toy problem, so that later the maths does not hide behind jargon."

The words, one by one

The slide's mini table: four loans, each with amount, period, credit score, and whether the borrower defaulted.

Amount taken	Period	Credit score	Defaulter
40 lakhs	5 years	1000	No
10 lakhs	5 months	550	Yes
80 lakhs	3 years	950	No
20 lakhs	4 years	1500	No

Training example	One row: an input $\mathbf{x}$ together with its true output $f(\mathbf{x})$. Written $\langle \mathbf{x}, f(\mathbf{x}) \rangle$.
Target function (target concept)	The <i>true</i> , unknown rule f that produced the labels — the real reason some borrowers default. We never see f ; we see only its values on the training rows.
Hypothesis	A proposed function h that we believe is similar to f . "Defaults if credit score < 600 " is a hypothesis.
Concept	A boolean target function. Rows with $f(\mathbf{x}) = 1$ are positive examples ; rows with $f(\mathbf{x}) = 0$ are negative examples .
Classifier	A discrete-valued function whose possible outputs $\{1, \dots, K\}$ are the classes or class labels.
Hypothesis space H	The set of <i>all</i> hypotheses the learning algorithm could, in principle, output. Choosing an algorithm is choosing H : all straight lines, all decision trees of depth 3, ...
Version space	The subset of H that has not yet been ruled out by any training example — every hypothesis still consistent with everything seen so far.

THINK OF IT LIKE

A detective. The target function is what really happened. Each hypothesis is a theory. The hypothesis space is every theory the detective is willing to consider. Each clue (training example) eliminates theories; the version space is the shrinking list of theories still consistent with all the clues. Learning is the narrowing of the version space.

The inductive learning hypothesis

BIG IDEA

Any hypothesis found to approximate the target function well over a sufficiently large set of training examples will also approximate the target function well over other, unobserved examples.

This is the leap of faith on which all of machine learning stands. We only ever see a sample; we assume that a rule which fits the sample well will fit the rest of the world well too. It is an *assumption*, not a theorem — and it fails precisely when the training set is not representative (Chapter 5) or the hypothesis is so flexible that it fits the sample's accidents (overfitting, Session 3–4). Everything we did in this book — cleaning, representative sampling, sealing a test set — is about making this assumption deserve our trust.

Inductive learning ("prediction") therefore means: given examples of a function $(X, F(X))$, predict $F(X)$ for new X . Depending on what F returns:

Kind of problem	$F(X)$ is	EnjoySport example
Classification	Discrete: $F(X) \in \{\text{Yes, No, Maybe}\}$	Will we enjoy the sport today? Yes/No
Regression	Continuous: $F(X) \in [20, 100]$	Predicted humidity, 60 / 75 / 70 / 45
Probability estimation	$F(X) \in [0, 1]$	$P(\text{EnjoySport} = \text{Yes}) = 0.95, 0.7, 0.5, 0.6$

The EnjoySport problem and how a hypothesis is written

This little dataset (Mitchell, Chapter 2) returns in Session 7–8, where you will build a decision tree on it. Meet it now.

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Mitchell writes a hypothesis as a vector of six constraints, one per attribute. Each slot is either a specific value ("Cold"), a "?" meaning *any value is acceptable*, or a "∅" meaning *no value is acceptable*.

- $\langle ?, \text{Cold}, \text{High}, ?, ?, ? \rangle$ "Enjoy the sport on any day that is Cold with High humidity, whatever the rest." One possible hypothesis.
- $\langle ?, ?, ?, ?, ?, ? \rangle$ The **most general** hypothesis: every day is a positive example.
- $\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$ The **most specific** hypothesis: no day is a positive example.

STOP AND THINK

Is $\langle ?, \text{Cold}, \text{High}, ?, ?, ? \rangle$ consistent with the four examples? (Consistent means: it says Yes exactly on the positive rows and No on the negative ones.)

OUR ANSWER

No. Example 3 is Cold and High, so this hypothesis says Yes — but the true label is No. And examples 1, 2, 4 are Warm, so it says No where the truth is Yes. It is inconsistent on all four rows and would be thrown out of the version space. A consistent one: $\langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, ?, ? \rangle$ — check it: Yes on rows 1, 2, 4 (all Sunny, Warm, Strong) and No on row 3 (Rainy, Cold). That is exactly what the Find-S algorithm in Mitchell would return, and it is also, in tree language, the rule "Sky = Sunny $\Rightarrow$ Yes".

EXAM TIP

Definitions of *hypothesis*, *hypothesis space*, *version space* and a statement of the *inductive learning hypothesis* are quick marks. For the last one, quote it and then add one sentence on when it fails (unrepresentative data; overfitting).

IN ONE BREATH

Target function f : the truth. Hypothesis h : a guess. Hypothesis space H : all allowed guesses. Version space: guesses not yet contradicted. Inductive learning hypothesis: a guess that fits enough training data will fit new data too — the assumption every model relies on.

CHAPTER 9

Challenges, Self-Check, and What Comes Next

The three challenges the slides flag for later, a twelve-question self-check, and the bridge to linear regression.

Three challenges of machine learning

The slides close with a preview that the later modules take up in detail. Know the names and one line each:

Challenge	Meaning	Where you meet the fix
Training data: insufficient	Too few examples for the pattern to show. (Challenge 1 in the WILP case study: trade off a fancier algorithm against how ready the data is.)	Learning curves, Session 3–4
Training data: non-representative	The sample does not look like the cases we will meet (Challenge 2). Small samples add noise; flawed sampling adds bias.	Stratified sampling — this book
Model selection: overfitting / underfitting	Model too flexible (memorises noise) or too simple (misses the pattern).	Bias–variance and regularisation, Session 3–4; pruning, Session 7–8
Validation and testing	Choosing settings and estimating true performance without cheating.	Train/validation/test — this book; metrics, Session 5–6

The WILP case study, in one paragraph

The running case on the slides: BITS WILP wants to **predict the Semester-3 GPA** of M.Tech AIML students to estimate how many will be eligible for the dissertation semester, so that accounts can bill partner companies. Attributes available: name, gender, age, date of birth, organisation, job title, nature of job, entrance score, eligibility score, previous degree, batch, section, marks in ISM/MFML/ACI/ML/NLP. The **observation** that makes it learnable: students with similar educational backgrounds tend to score similarly. Feature engineering as the slides show it: drop identifiers and irrelevant fields (name, gender, DOB, organisation, job title, batch, section), keep previous degree and the subject marks, and construct GPA / completion flags from raw totals. It is the whole book in miniature.

Twelve questions (cover the right column)

#	Question	Answer
1	Name the three components every ML algorithm has.	Data representation, parameter optimisation, model evaluation/selection.
2	Which operations are meaningful for an ordinal attribute?	Distinctness ($=$, $\neq$) and order ($<$, $>$); not differences or ratios.
3	Why is Celsius interval but Kelvin ratio?	Kelvin has a true zero, so ratios ("twice as hot") are meaningful; Celsius zero is arbitrary.
4	Give two reasons values go missing and three ways to handle them.	Not collected; not applicable. Eliminate rows/columns; estimate (impute); ignore during analysis.
5	IQR outlier rule?	Outlier if $x < Q_1 - 1.5 \text{ IQR}$ or $x > Q_3 + 1.5 \text{ IQR}$, with $\text{IQR} = Q_3 - Q_1$.
6	3-sigma rule and its assumption?	Outlier if outside $\mu \pm 3\sigma$; assumes an approximately normal distribution (99.7% inside).
7	When is a sample representative? Which scheme guarantees class proportions?	When it has approximately the same properties of interest as the full data. Stratified sampling.
8	Min-max formula; map 73,600 in [12,000, 98,000] to [0,1].	$v' = \frac{v - \min}{\max - \min}$; $61,600/86,000 = 0.716$.
9	Z-score formula; $\mu = 54,000$, $\sigma = 16,000$, $v = 73,600$.	$v' = (v - \mu)/\sigma = 19,600/16,000 = 1.225$.
10	Normalisation vs standardisation — when each?	Min-max when bounds known, data roughly uniform, no distribution assumption (kNN, NN); z-score when Gaussian assumed, range unbounded, outliers present.
11	Equal-width vs equal-depth binning?	Equal-width: intervals of size $(B - A)/N$; equal-depth: each interval holds $\approx n/N$ objects.
12	State the inductive learning hypothesis.	A hypothesis that approximates the target function well over a sufficiently large training set will also approximate it well on unobserved examples.

SPARK

Every technique in this book — typing attributes, cleaning, sampling, scaling, engineering features — has one purpose: to make the **inductive learning hypothesis** true for your data. A model can only generalise from data that is honest, representative, and expressed in features that carry the signal. Algorithms get the glory; preprocessing does the work.

STORY — TO BE CONTINUED

Meera's table is finally clean: no duplicates, one currency, imputed mileage, one-hot fuel types, everything z-scored, a stratified test set sealed in a separate file. She scatter-plots price against kilometres and sees a cloud that leans downward. "Now," says Arjun, "draw the best line through it." That is Book 3.